Machine Learning 1 (MLE501.22), FSB | Lecturer: PhD. Nguyen Thi Kim Truc

Deep Learning for Music Genre Prediction Using Log-Mel Spectrograms and CNNs

By Nguyễn Sỹ Hùng (25MSA33055)

Abstract. This report presents my final project for the Machine Learning 1 course. It is both an individual report and a project report, documenting the objectives, methodology, implementation, and results of the work.

The project established **a comparation between the solutions**: 1) **Mel-frequency cepstral coefficients (MFCC) + Convolutional Neural Network (CNN)** baseline vs 2) **LogMel + CNN** baseline – which actually compares 2 types of input data representation, targeting **music genre recognition** (a.k.a. music genre prediction) – **a multiclass classification problem – using CNN.** From there, I successfully built **a production-like prototype**: an end-to-end, reproducible workflow that starts from data collection and dataset analysis, transforms raw audio into training-ready data, trains and evaluates the model, and finally deploys the trained model in a demo application.

Keywords. *Music Genre Recognition, Log-Mel Spectrograms, CNN, Audio Signal Processing, Model Training and Evaluation*

Table of Content

I. **Introduction** 2
a. Project Overview and Problem Statement 2
b. Project Goals 2
c. Scope and Constraints 3
II. **Methodology and Experimental Setup** 3
III. **CNN Baselines: MFCC vs Log-Mel input presentations** 3
a. Overview of Dataset and Baselines 3
b. Model Architectures & Training Setup 4
c. Evaluation - Comparation Result 5
IV. Production-like Log-Mel CNN & Music Genre Prediction 6
a. Objectives 6
b. Data Collection, EDA, and Data Preprocessing 7
c. Model Architecture and Training Procedure 8
d. Results and Evaluation 11
e. Demonstration Applications - Consumer Layer 14
V. **Discussion and Reflection** 14
VI. **Summary and Conclusion** 15
References 15

I. Introduction

a. Project Overview and Problem Statement

Music genre recognition is a **challenging** task because a **"genre" is not defined in a fully exact or scientific way**. Although music contains measurable properties such as rhythm, tempo, beat, pitch, and frequency patterns that can be analyzed computationally, genre labels are also influenced by human perception, culture, place, time, and the fact that many songs blend multiple styles. As a result, a single song may reasonably belong to more than one genre, so genre labels should be treated as approximate categories rather than absolute truth.

In this project, music genre recognition is **approached as a practical supervised learning problem. The goal is to classify an n-second music segment, specifically a 15-second clip in the main experiments, into its dominant genre using deep learning**. In particular, the system is designed to predict the dominant genre label directly from audio data.

This problem is not easy for two main reasons. First, music is a complex signal with rich time-frequency structure. Second, genre boundaries are often unclear or overlapping, and short audio clips may not always contain enough information for perfect classification. Different genres can share similar acoustic patterns, while songs within the same genre can vary significantly in style.

To address this, the project converts audio clips into **Log-Mel spectrograms**, which provide compact time-frequency representations of sound, and **uses CNNs** to learn genre-related patterns from labeled training data. **The model does not attempt to define the true meaning of music genre; instead, it learns from the available labels and aims to make the best possible prediction of the dominant genre** for each input segment.

The project is mainly intended **for learning and practical exploration** rather than for producing a publishable research result or a fully commercial system. Its purpose is to **apply course knowledge, instructor guidance, and self-study to a realistic machine learning problem**, while strengthening understanding of data preparation, model design, training,

This report is accompanied by Python notebooks and scripts for model training, exploratory data analysis (EDA), and other supporting tasks.

b. Project Goals

- ✓ **Prepare the dataset, preprocessing pipeline, and exploratory data analysis (EDA)**
- ✓ Re-engineer the provided MFCC + CNN baseline for a better speed performance (the original training script for the MFCC + CNN baseline runs quite slow)
- ✓ Build a Log-Mel + CNN baseline with a similar setup for fair comparison
- ✓ **Build a more practical, production-like Log-Mel + CNN version**
- ✓ Compare and benchmark the different model versions
- ✓ Evaluate the results and analyze strengths and weaknesses
- ✓ **Develop a demo application**
- ✓ Explore whether a compact model can still be useful in practice

Overall, the project moves step by step from a basic instructional baseline toward a stronger proof-of-concept model.

c. Scope and Constraints

This project is limited by time, effort, hardware, and computing resources. It is a final course assignment, so its purpose is to show how the learned concepts can be applied in practice, not to build a full industrial product.

The project assumes a home-lab setup with a normal consumer GPU, not a large cloud or multi-GPU system. Training time should stay within a few hours, or at most around 24 hours, not multiple days. Because of this, the model must stay reasonably small and efficient.

Also, this is not a safety-critical system. The model is expected to classify at least eight common music genres, and occasional mistakes are acceptable because they do not cause serious harm. In practice, the model is meant to be a supportive tool for tasks such as tagging, browsing, or lightweight recommendation support, rather than a perfect decision-making system.

II. Methodology and Experimental Setup

This project follows a step-by-step experimental approach. It begins with dataset preparation, preprocessing, and exploratory data analysis (EDA). Audio clips are converted into fixed n-second segments and represented as log-Mel spectrograms for CNN-based training. For comparison, the project first uses the provided MFCC + CNN baseline trained on the small FMA dataset. Based on this reference, a new Log-Mel + CNN baseline is defined to remain as close as possible to the original model, with only minor adjustments required by the different input representation. This enables a more controlled comparison of how MFCC and log-Mel features affect model performance.

Model development then proceeds incrementally. After re-engineering and understanding the baseline, the project extends the work by building a custom dataset from several open datasets together with manually collected and processed audio clips. The model is then further improved into a more production-like version through practical refinements in preprocessing, architecture, and training design. This version is finally demonstrated through a proof-of-concept application. The models are trained, benchmarked, and evaluated under realistic personal computing constraints, and the results are analyzed to assess both their practical usefulness and their limitations.

All data processing, model training, and related tasks were implemented in Python using libraries and tools such as Librosa, FFmpeg, Sklearn, Pandas, TensorFlow + Keras, and others. The training pipeline also supports GPU acceleration for faster computation. In addition, metadata/manifests are stored in Parquet format for efficient data management.

III. CNN Baselines: MFCC vs Log-Mel input presentations

a. Overview of Dataset and Baselines

The baseline comparison in this project uses the FMA small subset prepared through manifest files. In the two notebooks reviewed, the loaded split contains

6,400 training rows, 800 validation rows, and 800 test rows across 8 genres: Electronic, Experimental, Folk, Hip-Hop, Instrumental, International, Pop, and Rock. After feature-cache filtering, 6,396 training items remained usable for both the MFCC and log-Mel experiments, while validation and test each remained at 800.

The MFCC baseline that is implemented in this project, is a modernized reimplementation of the official FMA MFCC→CNN baseline. It keeps the same core MFCC-as-2D-input idea and the same overall "not-too-deep" CNN design, but the actual implementation used in this project is not a byte-for-byte copy of the original baseline: it includes a refactored data pipeline, cached per-track features, training-set-based normalization, Batch Normalization layers, checkpointing, and a reproducible run-directory structure.

The Log-Mel baseline was developed as a controlled counterpart for comparison, using the same dataset split, the same optimizer family and main training settings, and the same evaluation protocol. However, it does not merely swap the input representation while keeping exactly the same network. Instead, the architecture was minimally adjusted to better match the 2D time-frequency structure of log-Mel spectrograms.

Therefore, the comparison in these baselines should be understood as a comparison between two closely related "not-too-deep" CNN baselines built on the same FMA split and training recipe, but with different audio representations and slightly different architectural choices to better suit each representation.

b. Model Architectures & Training Setup

MFCC CNN baseline	Log-Mel CNN baseline
Input (13, 2582, 1)	**Input** (192, 2582, 1)
Conv2D(3, kernel=13×10, stride=1×4)	**Conv2D**(16, kernel=5×5, stride=2×2)
BatchNorm + **ReLU** → (1, 644, 3)	**BatchNorm** + **ReLU** → (64, 1291, 16)
Conv2D(15, kernel=1×10, stride=1×4)	**Conv2D**(32, kernel=3×3, stride=2×2)
BatchNorm + **ReLU** → (1, 159, 15)	**BatchNorm** + **ReLU** → (32, 646, 32)
Conv2D(65, kernel=1×10, stride=1×4)	**Conv2D**(64, kernel=3×3, stride=2×2)
BatchNorm + **ReLU**→ (1, 38, 65)	**BatchNorm** + **ReLU** → (16, 323, 64)
Flatten → (2470,)	**GlobalAveragePooling2D** → (64,)
Dense(8, **Softmax**) → (8,)	**Dense**(8, **Softmax**) → (8,)
Total params: 30,690	Total params: 24,408

Both models are trained on the same FMA small split with batch size 16, up to 99 epochs, and early stopping. They also share the same training setup: Adam optimizer with learning rate 0.0003, categorical cross-entropy loss, accuracy as the training metric, and ModelCheckpoint based on validation loss.

The MFCC baseline takes input of shape **(13, 2582, 1)** and follows a "not-too-deep" FMA-style design: three convolutions that collapse the MFCC axis early, followed by Flatten and a softmax classifier. In the notebook actually used here, each convolution is followed by Batch Normalization and ReLU, making the baseline more stable while preserving its original idea.

The log-Mel baseline takes input of shape **(128, 2582, 1)** and uses a "not-too-deep" CNN better suited to spectrograms: three true 2D convolutions with kernel sizes

5×5, 3×3, and 3×3, all with stride **2×2**, each followed by Batch Normalization and ReLU, then Global Average Pooling and a softmax classifier. So, unlike the MFCC model, it does not collapse the full frequency axis in the first layer.

This difference is important. For MFCCs, early collapse of the coefficient axis is reasonable because MFCCs are already a compact summary of spectral shape. For log-Mel spectrograms, the frequency axis still contains meaningful local structure, so local 2D kernels provide a better inductive bias.

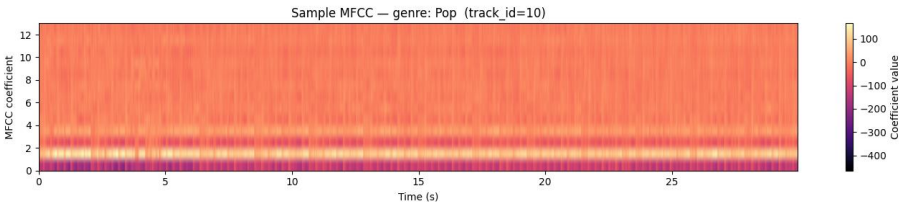
Therefore, the two baselines do not differ only in input representation. They share the same data split, optimizer, loss, batch size, epoch budget, and evaluation procedure, but their CNN architectures are only similar in spirit, not identical. Even so, the comparison remains valid because the log-Mel model was kept small and "not-too-deep", with only limited architectural changes to better fit log-Mel input. Thus, it is more accurate to describe it as a **minimally adapted "not-too-deep" 2D CNN baseline for log-Mel spectrograms**, rather than an exact copy of the MFCC baseline.

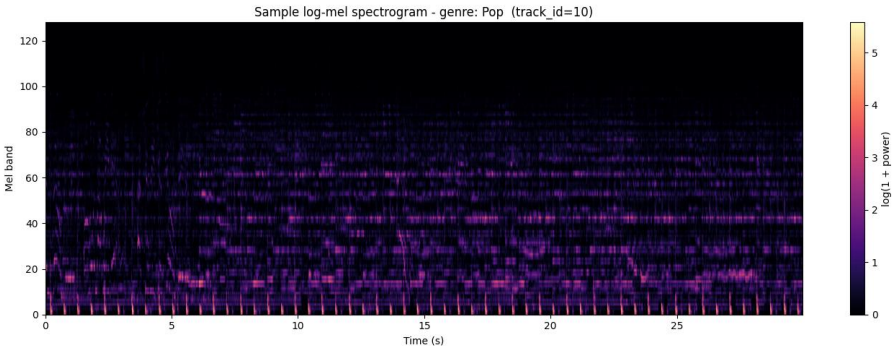
c. Evaluation - Comparation Result

Both baselines use the same dataset split configuration:

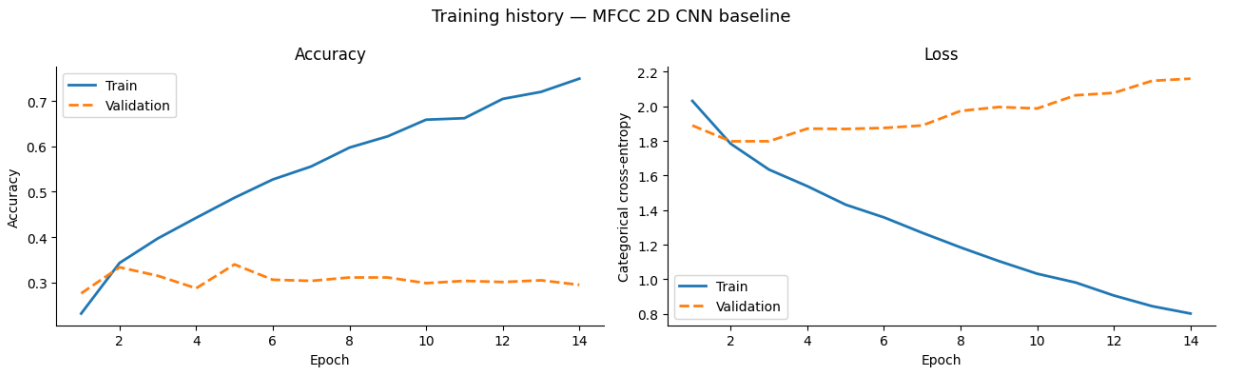
Split	Samples	Share
Train	6,400	80%
Validation	800	10%
Test	800	10%

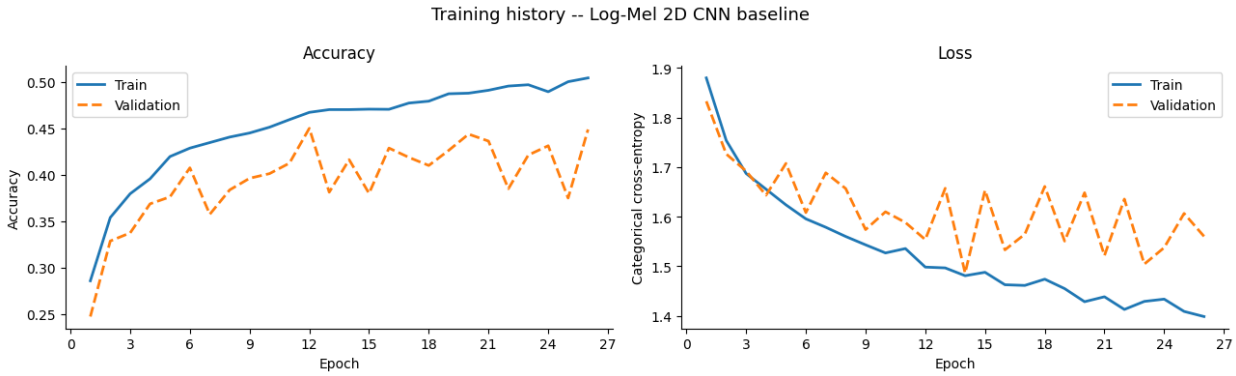
- Total samples: 8,000
- Genres / classes: 8
- Genre labels: Electronic, Experimental, Folk, Hip-Hop, Instrumental, International, Pop, Rock





Training History:





Early stopping behavior: MFCC stopped at epoch **14**, Log-Mel stopped at epoch **26**. This suggests the Log-Mel model was able to continue improving for longer before validation performance stopped improving. That is another sign that the Log-Mel representation gave the model a more learnable signal.

Test result:

Model	Test Accuracy	Test Macro-F1	Test Loss
MFCC + CNN	27.38%	0.2711	1.9227
Log-Mel + CNN	39.25%	0.3843	1.6647

Compared with MFCC, the Log-Mel model improves by about **+11.88 percentage points** in test accuracy, **+0.1132** in test Macro-F1, and **lower test loss by 0.258.**

Validation set tells the same story:

Model	Val Accuracy	Val Macro-F1
MFCC + CNN	33.38%	0.3132
Log-Mel + CNN	41.62%	0.4070

The MFCC model shows a **much larger drop from train to test**, which means it is **less stable and less transferable** to unseen data. The Log-Mel model has a **smaller generalization gap**, so it is not only more accurate, but also **more reliable**.

The Log-Mel + CNN baseline clearly **outperforms the MFCC + CNN** baseline on the benchmark.

IV. Production-like Log-Mel CNN & Music Genre Prediction

a. Objectives

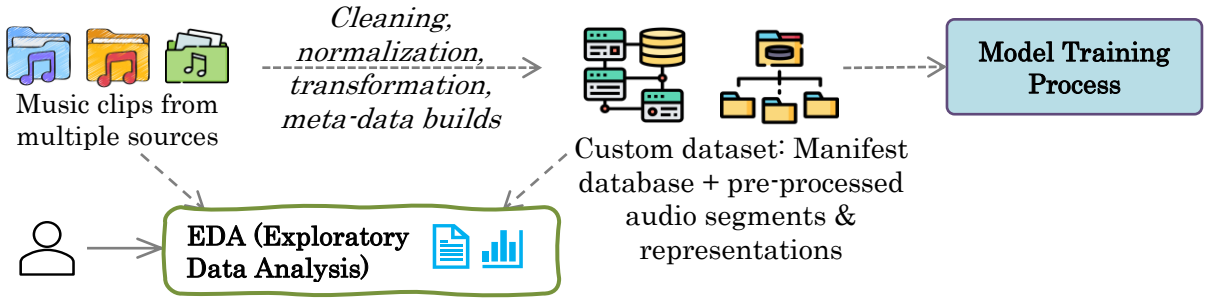
This part of the project aims to develop a small **production-like music genre classification model** based on **log-Mel spectrograms and a CNN architecture**. The

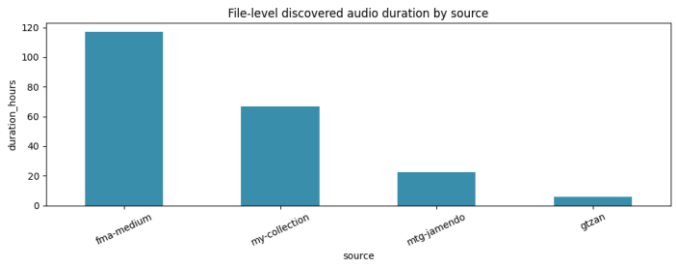
objective is to move from a simple research baseline toward a model that is more practical for real-world use.

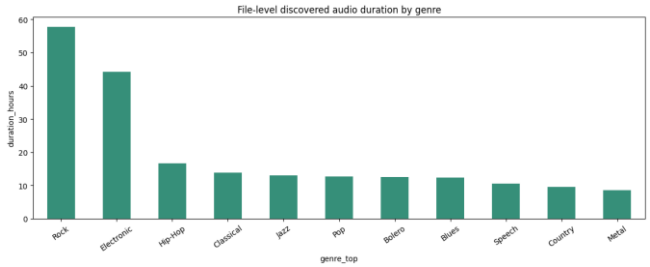
Since no single existing music dataset that I could identify fully satisfied my requirements, I constructed a custom dataset by selectively combining samples from multiple open datasets with audio files from my own music collection. The resulting dataset contains **nearly 300,000 audio clips**, each standardized to **15 seconds**, and initially focuses on **10 target classes (genres): Hip-Hop, Pop, Rock, Metal, Classical, Jazz, Country, Blues, Bolero (Việt Nam),** and **Speech.**

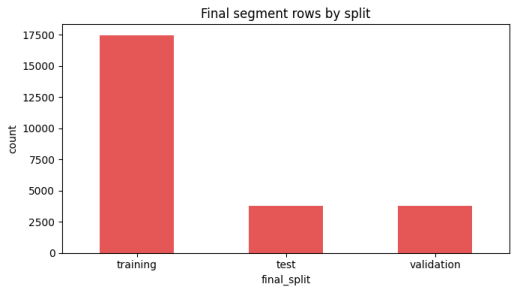
The performance target for this model is to achieve **at least 80% Macro-F1 and Accuracy**, while keeping the overall training process **practical on mid-range personal computing hardware**, supported by **CUDA-enabled GPU acceleration**.

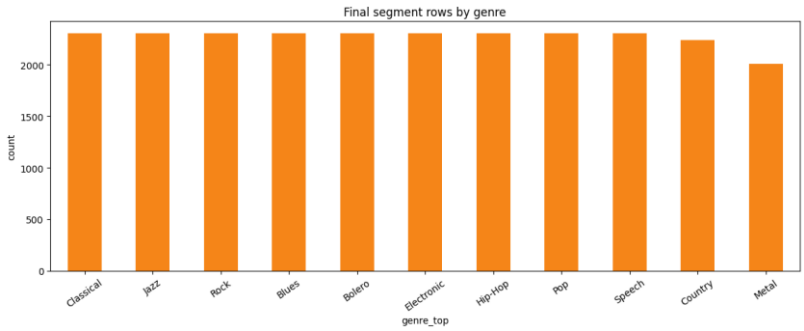
b. Data Collection, EDA, and Data Preprocessing











	count	mean	median	mode	std	variance	min	max	skewness	kurtosis
segments_per_file	11336	2.00	2.00	2.00	0.01	0.00	1.00	2.00	-106.47	11336
source_file_duration	11336	30.78	31.00	31.00	0.41	0.17	24.06	31.00	-1.71	5.89
unique_files/genre	10	1133.60	1151.00	1151.00	46.25	2139.16	1005.00	1152.00	-2.93	8.75
coverage_ratio_pct	11336	97.46	96.77	96.77	1.36	1.85	62.33	100.23	-0.25	38.74
utilization_ratio	11336	1.09	1.00	1.00	0.28	0.08	1.00	2.00	2.96	6.74

Training dataset summary:

Total rows	**22,028** (total duration = 15s * 22028 ~ **92 hours**)
Train (~70%)	15,4100020rows
Validation	3,309 rows
Test	3,309 rows
Number of classes	**10**
Genres	Blues, Bolero, Classical, Country, Hip-Hop, Jazz, Metal, Pop, Rock, Speech
Sample length	**15.0s**

c. Model Architecture and Training Procedure

This section describes the Log-Mel input representation, model architecture, training procedure, and related enhancement techniques. The model classifies music genres from audio clips using log-Mel spectrograms instead of raw waveforms. Since a log-Mel spectrogram is a 2D time–frequency representation, it is treated as a single-channel image and processed with a 2D CNN.

Input Representation and Normalization

A spectrogram $X \in \mathbb{R}^{(B\times T)}$, where B is the number of Mel bins and T is the number of time frames. Given an audio clip, it is first converted into a log-Mel spectrogram, which can be written as:

$$X[b,t] = \log(1 + M_\beta[t])$$

where $M_\beta[t]$ is the Mel-band energy at Mel bin b and time frame t, computed by applying a **Mel filterbank** to the **power spectrum** (STFT/FFT) of the signal:

$$M_\beta[t] = \Sigma_{k=0}^{K_fft-1} H_\beta[k] \cdot P_t[k]$$

Before training, the training script standardizes the spectrograms using **training-set-only statistics**. For each Mel bin b, it computes a mean μ_β and standard deviation σ_β, then applies:

$$\tilde{X}[b,t] = (X[b,t] - \mu_\beta) / (\sigma_\beta + \varepsilon)$$

This makes the input scale more consistent and helps the model train more stably.

Data Augmentation. During training, the script applies two main augmentation methods.

- **Mixup** blends two training samples and their labels: $X' = \lambda X_i + (1-\lambda)X_j$. This encourages smoother decision boundaries and reduces overfitting. It can be thought of something like "this is mostly Rock, but slightly influenced by Country - don't be too confident". This helps force the model to not memorize exact training spectrograms as it sees blended versions, giving a softer prediction instead of a sharp all-or-nothing guess. In this way, it improves generalization. Also, real songs often contain overlapping genre traits, so Mixup helps the model more align with the nature of data.
- **The use of SpecAugment** in this project helps improves robustness by randomly hiding small parts of the log-Mel spectrogram during training, forcing the model to recognize genre from broader and more distributed evidence instead of memorizing a few fragile local cues.

Model Architecture. The model is composed of five convolutional blocks followed by a lightweight classifier head. In compact form, the architecture is: **Input→ Conv Blocks → Global Average Pooling Dense → Softmax**

The architecture follows a standard CNN design principle:

- Early layers learn **local spectral patterns** (e.g., timbre, harmonics)
- Deeper layers capture **global and abstract structures** (e.g., rhythm, genre characteristics)
- GAP + Dense layers convert these learned features into **final genre predictions**

Model Layer Details

```
L0) Input → (128, 861, 1)
L1) 5 Conv blocks
→ Block 1: Conv(32, 5×5) + BN (BatchNormalization) + ReLU + MaxPool (2×2)
→ Block 2: Conv(64, 3×3) + BN + ReLU + SpatialDropout(0.05) + MaxPool(2×2)
→ Block 3: Conv(128, 3×3) + BN + ReLU + SpatialDropout(0.10) + MaxPool(2×2)
→ Block 4: Conv(256, 3×3) + BN + ReLU + SpatialDropout(0.15) + MaxPool(2×2)
→ Block 5: Conv(256, 3×3) + BN + ReLU + SpatialDropout(0.20) + MaxPool(2×2)
L2) GlobalAveragePooling2D → (256)
L3) Dense(256) + ReLU
L4) Dropout(0.20)
L5) Dense(n_classes, Softmax) → class probabilities
```

Each convolutional block follows the pattern: **X** → Conv2D → BatchNorm → ReLU → (SpatialDropout) → MaxPool:

- **Batch Normalization** stabilizes training and improves convergence
- **ReLU** introduces non-linearity
- **SpatialDropout2D** (blocks 2–5) regularizes feature maps by dropping entire channels
- **Max Pooling** reduces spatial dimensions and increases receptive field

The number of filters increases across blocks 32 → 64 → 128 → 256 → 256. At the same time, repeated pooling progressively reduces the spatial resolution of the

feature maps, enabling the model to transition from low-level time–frequency patterns to higher-level, genre-discriminative features.

After the convolutional backbone, we have the following called Classifier Head, the final part of the network that takes learned features and turns them into class predictions:

- **Global Average Pooling (GAP)** collapses spatial dimensions and summarizes each feature map into a single value
- A **Dense(256)** layer acts as a bottleneck representation
- **Dropout (0.20)** provides additional regularization
- The final **Dense(n_classes, softmax)** layer outputs class probabilities

<u>Global Feature Aggregation (GAP) and Output</u>

After the last convolution block, the model uses **global average pooling**. Instead of flattening the whole feature map, it averages each channel:

$$g_c = \frac{1}{HW} \sum_{u=1}^{H} \sum_{v=1}^{W} F_c\,(u, v)$$

This produces a compact feature vector, which is passed through a dense layer and then a final output layer. GAP keeps the important "what features exist" information while avoiding the huge parameter cost and overfitting risk that Flatten would create. Using Flatten makes the classifier head becomes very large and may start memorizing training data instead of learning robust genre structure. GAP keeps the head compact, which is usually safer.

The output layer uses **softmax**, so the predicted probability for class c is: $p_c = \frac{e^{z_c}}{\sum_j e^{z_j}}$

The final prediction is: $\hat{y} = \arg\max_c p_c$

<u>Model Training</u>

The model is trained using **categorical cross-entropy**: $\mathcal{L} = -\sum_c y_c \log p_c$

Because Mixup is used, the target y may be a soft label rather than a strict one-hot vector.

The optimizer is **AdamW**, which combines adaptive gradient updates with weight decay. The learning rate uses **warmup followed by cosine decay**, helping training begin smoothly and then gradually refine the model.

At each training step, a batch of spectrograms is loaded, normalized, augmented, and passed through the model. The loss is computed, gradients are obtained by backpropagation, and the parameters are updated. In simplified form: $X \to \tilde{X} \to f_\theta(\tilde{X}) = p \to \mathcal{L}(y, p)$

Then the parameters θ are updated to reduce the loss. Model selection is based mainly on **validation macro-F1**, not only accuracy. This is useful because macro-F1 gives equal importance to every class.

<u>Inference</u>

At inference time, the process is simpler. A new audio sample is converted into a log-Mel spectrogram, normalized with the same training-set statistics, and passed through the trained model: $X \to \tilde{X} \to f_{\theta^*}(\tilde{X}) \to \hat{y}$. The output is a probability distribution over genres, and the class with the highest probability is returned.

d. Results and Evaluation

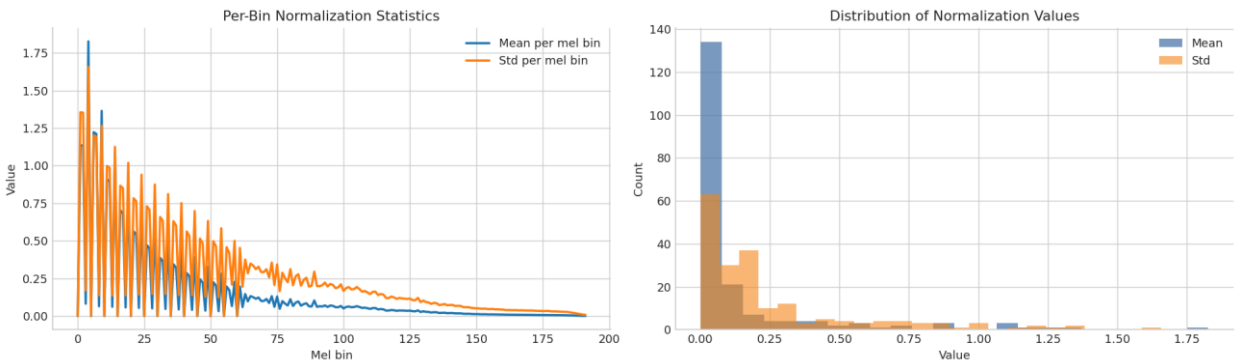
This section summarizes the final training run of **Log-Mel CNN**, with emphasis on the evaluation results and their interpretation. The model was trained on a **10-class music and speech classification dataset** of **22,028 samples**, split into **15,410 training**, **3,309 validation**, and **3,309 test** samples (for more details, please see section IV.b above). Inputs were represented as **log-Mel spectrograms** of shape **(192, 1291, 1)**, generated from **15-second audio clips** at **22,050 Hz**.

The training setup used **AdamW**, **cosine learning-rate annealing with warm-up**, **batch size 48**, and **early stopping**. To improve generalization, the pipeline combined **Mixup**, **SpecAugment**, **SpatialDropout2D**, **Dropout**, **label smoothing**, **gradient clipping**, and **class weighting**. Training stopped after **85 epochs** out of a maximum of **136**, and the final model weights were restored according to the early stopping policy.

Overview of dataset and training setup

Item	Value
Total samples	22,028
Train / Validation / Test	15,410 / 3,309 / 3,309
Number of classes	10
Input representation	Log-Mel spectrogram
Input shape	(192, 1291, 1)
Clip duration	15.0 s
Sample rate	22,050 Hz
Optimizer	AdamW
Batch size	48
Max / actual epochs	136 / 85
Main regularization	Mixup, SpecAugment, SpatialDropout2D, Dropout, label smoothing
Normalization	Train-only per-Mel-bin standardization

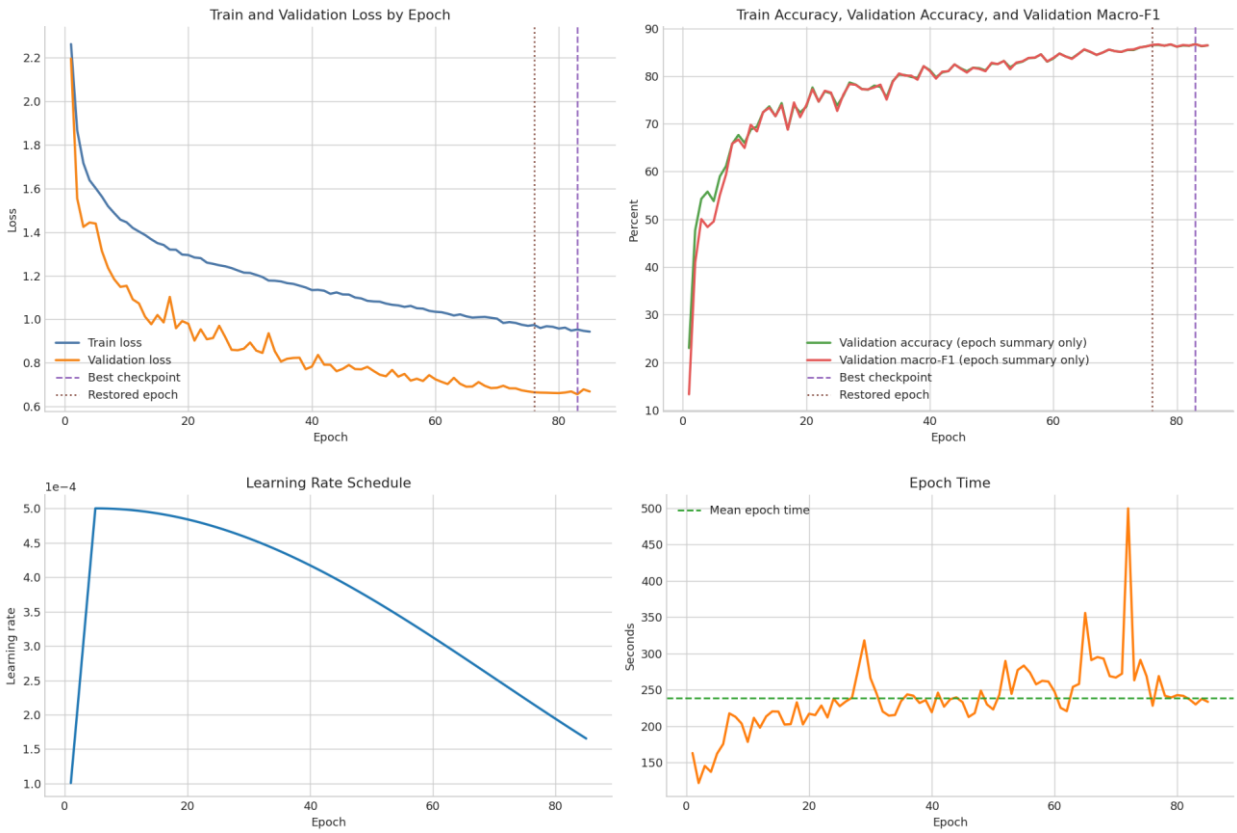
Normalization Diagnostics:



Regarding the per-Mel-bin normalization diagnostics plot above, the left panel shows that lower Mel bins have larger means and standard deviations, while higher bins gradually decrease, confirming that different Mel bands operate at different scales. The right panel shows right-skewed distributions of these values, with most bins concentrated at smaller values and a smaller subset much larger.

Overall, the diagnostics indicate that the train-only per-bin normalization statistics are stable and appropriate for reuse.

Training progress:



Overall evaluation results:

Split	Loss	Accuracy	Macro-F1
Train	0.4369	95.70%	0.9572
Validation	0.6559	86.70%	0.8667
Test	**0.6597**	**87.07%**	**0.8695**

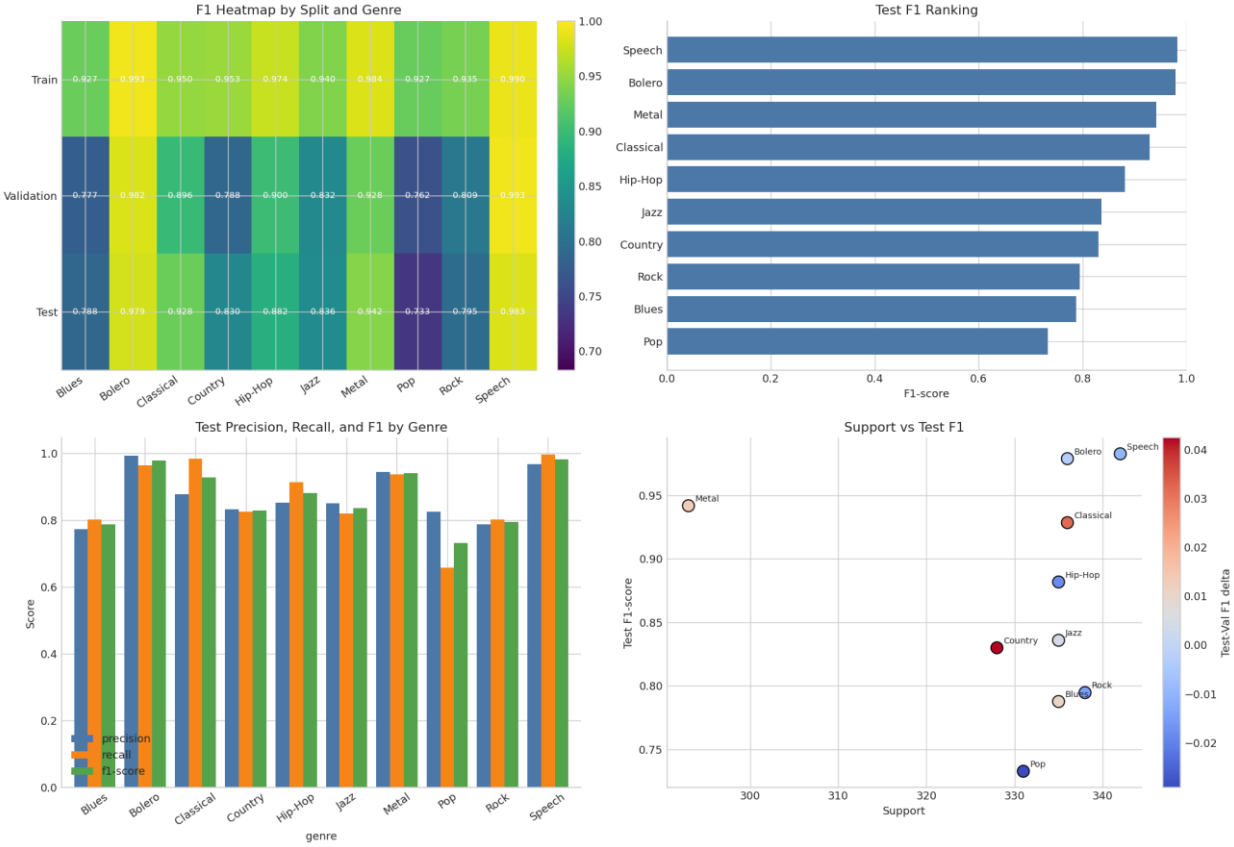
The final evaluation results show that the model performed strongly on unseen data. It achieved **87.07% test accuracy**, **0.8695 test Macro-F1**, and **0.6597 test loss**. Validation performance was very close, at **86.70% accuracy** and **0.8667 Macro-F1**, which indicates that the selected model generalized well. Training performance remained higher, with **95.70% accuracy** and **0.9572 Macro-F1**, showing that some overfitting still existed, but the train–test gap was moderate rather than severe. In bias-variance terms, it looks like low bias with moderate variance which is normal in this case, and the variance is still under control. In short, it might be said that mildly to moderately overfit in the strict sense but **not overfit in a way that appears harmful for deployment-quality generalization**

From these results, two main conclusions can be drawn. First, the model achieved the project target of strong classification performance while remaining compact and practical for consumer-hardware training. Second, the close validation and test scores suggest that the training strategy and regularization design were effective in controlling overfitting.

Per-class results show that the model performed best on **Speech (F1 = 0.9827)**, **Bolero (0.9789)**, **Metal (0.9418)**, and **Classical (0.9285)**. These classes are likely easier to recognize because they have more distinctive acoustic patterns. In contrast, the weakest classes were **Pop (0.7328)**, **Blues (0.7877)**, and **Rock (0.7947)**, indicating that these genres remain more difficult to separate.

Per-class test results:

Genre	Precision	Recall	F1-score
Speech	0.9688	0.9971	0.9827
Bolero	0.9939	0.9643	0.9789
Metal	0.9450	0.9386	0.9418
Classical	0.8780	0.9851	0.9285
Hip-Hop	0.8524	0.9134	0.8818
Jazz	0.8514	0.8209	0.8359
Country	0.8338	0.8262	0.8300
Rock	0.7878	0.8018	0.7947
Blues	0.7730	0.8030	0.7877
Pop	0.8258	0.6586	0.7328



These class-level results suggest that the remaining errors are concentrated in genres with stronger stylistic overlap. In particular, **Pop**, **Rock**, and **Blues** share similar instrumentation, rhythm, and production characteristics, making their boundaries less distinct. Therefore, some classification errors likely reflect the nature of the genre-labeling problem itself, rather than only weaknesses in the model.

Overall, the final run validates the project's production-like direction. The model achieved high held-out performance, stable convergence, and practical training efficiency, while its remaining weaknesses are concentrated mainly in naturally

overlapping genres. This supports the conclusion that a **compact, well-regularized Log-Mel CNN**, combined with a reproducible training pipeline, can serve as an effective and realistic solution for music genre classification.

e. Demonstration Applications - Consumer Layer

The demonstration application is a consumer-facing layer for near-real-time music genre recognition. It includes a Flask-based local client that captures system audio via WASAPI loopback, maintains a rolling buffer, converts the stream into the model's inference format, and sends audio to the inference service for live prediction. It bridges the trained production-like Log-Mel CNN model and an interactive browser UI, allowing users to view real-time genre predictions, confidence scores, and session status.

Built on top of the project's broader pipeline – dataset creation, log-Mel feature generation, TFRecord conversion, model training, and model serving – the app is not just a standalone demo, but the presentation layer of the end-to-end system for practical streaming validation and interactive demonstration.

V. Discussion and Reflection

This project taught me that model improvement comes from systematic iteration rather than one major change. I learned that data quality and diversity often matter more than model complexity, and that the input representation must align with the CNN design. I also learned that regularization techniques such as Mixup, SpecAugment, dropout, and class weighting only help when balanced carefully.

Another important lesson was that some classification errors reflect the nature of the problem itself, since several music genres overlap in style. Finally, I learned

that the surrounding pipeline – data preparation, normalization, TFRecord conversion, and inference engineering – is just as important as the model for achieving strong practical results.

VI. Summary and Conclusion

The Log-Mel CNN baseline implemented in this project outperforms the MFCC CNN baseline, which supports the hypothesis that Log-Mel spectrograms provide richer and more discriminative information by preserving both temporal and frequency structure in an image-like representation. Compared with the Log-Mel CNN baseline, the production-like model achieves significantly better performance and is considerably more practical for real-world deployment.

This project delivered a complete, reproducible pipeline for deep learning-based music genre classification, covering data preparation, log-Mel feature extraction, TFRecord conversion, model training, evaluation, and real-time inference. Through a sequence of carefully analyzed model improvements, performance increased from a weak MFCC baseline to a strong production-like LogMel CNN, reaching about 0.8707 accuracy as a proportion, and 0.8695 test macro-F1 on 10 genres. The final outcome demonstrates that, even under consumer hardware and course-project constraints, a compact and well-regularized CNN can achieve practical results when supported by good data engineering, appropriate architecture design, and systematic experimentation.

Thank you!

Nguyen Sy Hung,

Mar. 26th, 2026

References

Nguyen Thi Kim Truc. (2026). Machine Learning 1 (MLE501.22): Lecture materials and instructor guidance.

Tzanetakis, G., & Cook, P. (2002). Musical genre classification of audio signals. IEEE Transactions on Speech and Audio Processing, 10(5), 293–302. A classic foundational paper for automatic music genre classification.

Defferrard, M., Benzi, K., Vandergheynst, P., & Bresson, X. (2017). FMA: A dataset for music analysis. In Proceedings of the 18th International Society for Music InformationRetrieval Conference (ISMIR).

Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2020). Mathematics for Machine Learning. Cambridge University Press

Lerch, A., Arthur, C., Pati, A., & Gururani, S. (2024). An Introduction to Audio Content Analysis: Applications in Signal Processing and Music Informatics. Wiley.

Aggarwal, C. C. (2023). Neural Networks and Deep Learning: A Textbook (2nd ed.). Springer.